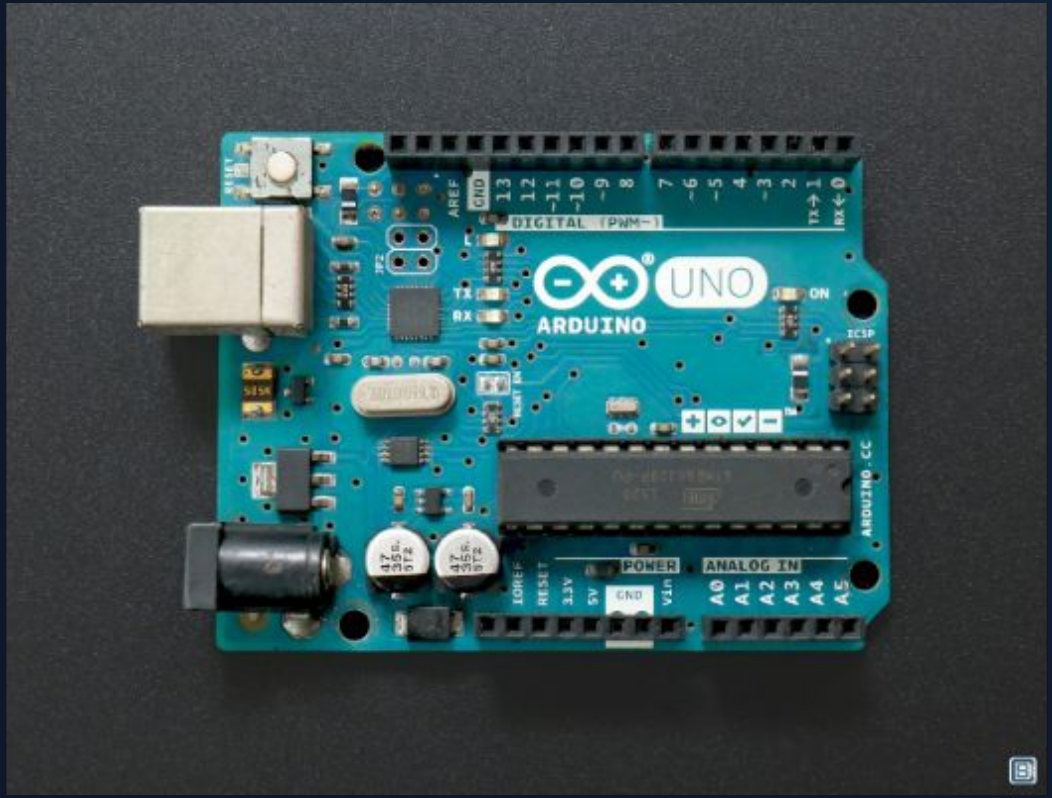
Computação Embarcada

Aula 02: Fundamentos C++, Memória e Estruturas de Controle

Parte 1: Revisão e Arquitetura

Revisão: Sistemas Embarcados

- **O que são?** Sistemas computacionais dedicados a funções específicas, interagindo continuamente com o mundo físico via sensores (entrada) e atuadores (saída).
- **Microcontrolador (MCU):** Diferente de um microprocessador (como o Core i7), um MCU possui CPU, ROM, RAM e periféricos unificados no mesmo silício.
- **Arquitetura Harvard:** O ATmega328P separa fisicamente os barramentos de Dados e Instruções. Isso permite a busca de uma instrução enquanto a anterior ainda está sendo executada (pipelining básico).
- **Estrutura do Sketch:** setup() para configurações iniciais (roda uma vez) e loop() para o ciclo infinito de controle.



O Mapa de Memória do ATmega328P

Memória Flash (32 KB)

Onde o seu código compilado (sketch) é armazenado. É uma memória não-volátil (mantém os dados sem energia). Possui vida útil de cerca de 10.000 ciclos de escrita/gravação.

SRAM (2 KB)

Memória de Acesso Aleatório Estática. É aqui que o MCU armazena as **variáveis dinâmicas** criadas durante a execução. Apenas 2048 bytes disponíveis! Otimização é crucial.

EEPROM (1 KB)

Memória não-volátil programável. Utilizada para salvar configurações de longo prazo ou registros do usuário (ex: senhas, contadores de inicialização). Sobrevive a reinicializações.

Parte 2: Dados e Sintaxe C++

Variáveis, Tipos, Escopo e Comunicação Serial

Escopo de Variáveis: Stack vs Heap

Onde as variáveis "vivem"?

No C++, o escopo define o tempo de vida e a visibilidade de uma variável. Na restrita SRAM do Arduino, gerenciar isso é vital.

- **Globais:** Declaradas fora de funções. Vivem para sempre. Facilitam o acesso, mas consomem RAM o tempo todo e aumentam o risco de bugs (efeitos colaterais).
- **Locais:** Declaradas dentro de funções ou blocos (ex: dentro de um if). São criadas na *Stack* e destruídas assim que a função termina, liberando RAM instantaneamente.

A palavra-chave `static`

E se quisermos uma variável local que não morra entre as execuções da função? Usamos o modificador static.

```
void loop() {
  static int contador = 0;
  contador++;
  // 'contador' é inicializado em 0 apenas uma vez.
  // Seu valor é retido (1, 2, 3...) nos próximos ciclos.
}
```

É a alternativa elegante e segura para evitar variáveis globais em contadores de estado e timers.

Tipos de Dados no ATmega328P

Tipo de Dado	Tamanho (Memória)	Faixa de Valores (Range)	Uso Típico
bool	1 byte (8 bits)	`true` (1) ou `false` (0)	Estados de botões, flags lógicas.
byte / uint8_t	1 byte (8 bits)	0 a 255	Mapeamento de pinos, contagens pequenas.
int / int16_t	**2 bytes (16 bits)**	-32.768 a 32.767	Leitura de sensores analógicos (ADC), cálculos base.
long / int32_t	4 bytes (32 bits)	-2.147.483.648 a +2.147.483.647	Cronômetros (`millis()`), cálculos grandes.
float	4 bytes (32 bits)	Ponto flutuante precisão simples	Sensores de temperatura, GPS, cálculos complexos.

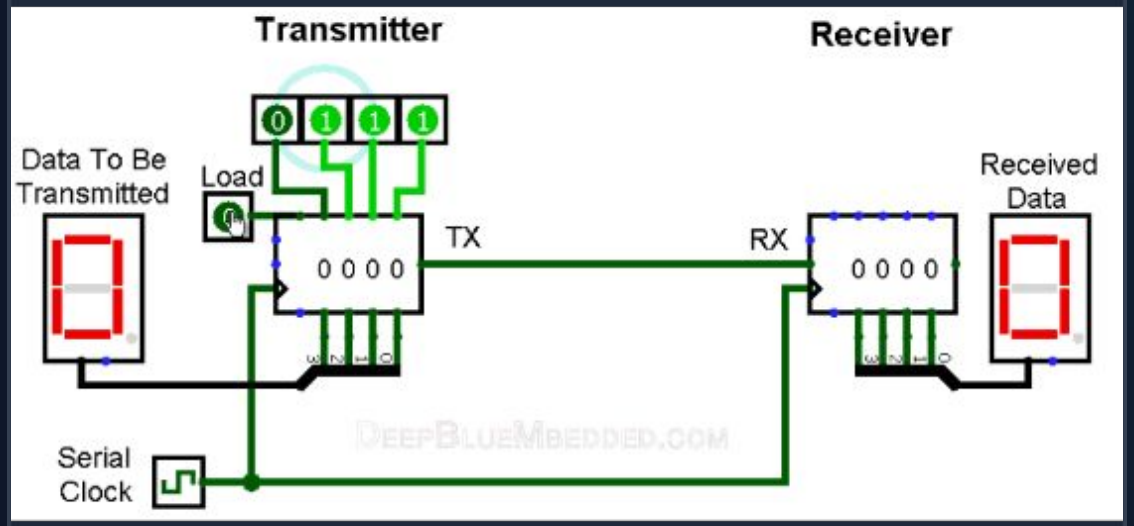
Nota Arquitetural: O ATmega não possui FPU (Floating Point Unit) em hardware. Cálculos com `float` são emulados via software, tornando a execução dezenas de vezes mais lenta. Evite `float` se puder usar inteiros multiplicados (ex: 25.4ºC armazenado como int 254).

Comunicação Serial (UART)

O que é o Monitor Serial?

O Arduino se comunica com o computador através do protocolo **UART** (Universal Asynchronous Receiver-Transmitter) usando os pinos TX (Transmissão) e RX (Recepção).

- **Baud Rate:** A velocidade da conversa. Serial.begin(9600); significa transmitir 9600 bits por segundo. Emissor e receptor devem estar perfeitamente sincronizados.
- **Envio (MCU → PC):** Serial.print() e Serial.println() enviam caracteres ASCII para o console. Útil para depurar o código "printando" valores.
- **Recepção (PC → MCU):** Serial.available() verifica quantos bytes chegaram fisicamente ao buffer da placa. Serial.read() lê os bytes em fila.



Parte 3: Lógica e Operadores

O Motor de Decisão do Microcontrolador

Operadores Clássicos

Aritméticos

Soma (+), Sub (-), Mult (*), Div (/), Resto (%).

Atenção: Divisão entre inteiros (ex: 5 / 2) trunca a casa decimal (resultado = 2 em C++). Use 5.0 / 2.0 para forçar o float.

Relacionais e Lógicos

Igualdade (==), Diferença (!=), Maior/Menor (>, <, >=, <=).

Lógicos unem múltiplas expressões: E (&&), OU (||), NÃO (!). Essenciais na avaliação de cenários de sensores.

Atribuição Composta

Atalhos geniais que reduzem ciclos de CPU: +=, -=, *=, /=.

Ex: x += 5;. Incrementos e decrementos também são vitais: x++ e x--.

Operadores Bit a Bit (Bitwise)

Por que usar Bitwise?

Em Computação Embarcada, precisamos manipular um único pino de hardware dentro de um registrador de 8 bits (ex: PORTB), sem alterar a polaridade dos outros 7 pinos vizinhos.

- **& (AND):** Usado para zerar (limpar / aplicar mask) bits específicos.
- **| (OR):** Usado para setar (ligar / forçar HIGH) bits.
- **^ (XOR):** Usado para inverter (toggle) o estado atual.
- **~ (NOT):** Inverte todos os bits de 0 para 1 e vice-versa.

Deslocamento de Bits (Shift)

Movem os bits na memória para a esquerda (multiplicando por base 2) ou direita (dividindo). São cálculos baseados em hardware ultra-rápidos.

```
byte x = 0b00000001; // Decimal 1
x = x << 3;            // Shift Left 3 vezes
// x agora é 0b00001000 (Decimal 8)

// Ligando apenas o Pino 5 físico do registrador:
PORTB = PORTB | (1 << 5);
```

Tomadas de Decisão

if / else if / else

A espinha dorsal da lógica embarcada. O sistema desvia o fluxo se a expressão dentro dos parênteses retornar **VERDADEIRO** (qualquer valor > 0).

```
if (temperatura > 30.0) { acionarExaustor(); } else if
(temperatura < 15.0) { acionarAquecedor(); } else {
modoEconomia(); }
```

switch / case e Ternário

O bloco switch é mapeado na compilação para tabelas de salto (Jump Tables), sendo muito eficiente para gerenciar Menus Físicos ou Máquinas de Estados Finitos. O break previne o vazamento para o próximo bloco.

O **Operador Ternário** (?:) é excelente para atribuições condicionais diretas:

```
// condicao ? caso_true : caso_false;  bool isAlerta =
(pressao > 100) ? true : false;
```

Laços de Repetição (Loops)

for

Ideal quando você sabe o número exato de iterações. Excelente para varrer Arrays de sensores, acender Matrizes de LEDs ou configurar múltiplos pinos na função setup() em lote.

while

Executa o bloco *enquanto* a expressão booleana for verdadeira. Muito utilizado em embarcados para interromper o programa aguardando um evento físico no pino.
Ex: while(botaoPressionado == 0) { }

do / while

Variação técnica do while onde a verificação de continuidade é feita no final do loop. Garante que o motor/algoritmo será executado **pelo menos uma vez** antes de ser testado.

Referências Bibliográficas

MCROBERTS, M. Arduino básico. 2. ed. São Paulo: Novatec, 2015. 506 p. ISBN 9788575224045.

MONK, S. Programação com Arduino: começando com sketches. 2. ed. Porto Alegre: Bookman, 2017. ISBN 9788582604472.

DEITEL, H. M.; DEITEL, P. J. C: Como programar. São Paulo: Pearson, 2011. 848 p. ISBN 9788576059349.

SOUZA DE LOPES, J. A. Lógica Matemática. 1. ed. São Paulo: Pearson, 2016.

WEBB, B.; CONSI, T. R. Biorobotics: methods and applications. MIT Press, 2001. 208 p. ISBN 026273141X.

Hora da Mão na Massa!

Prof. MSc. Diego Fernando Maia | dmaia@up.edu.br